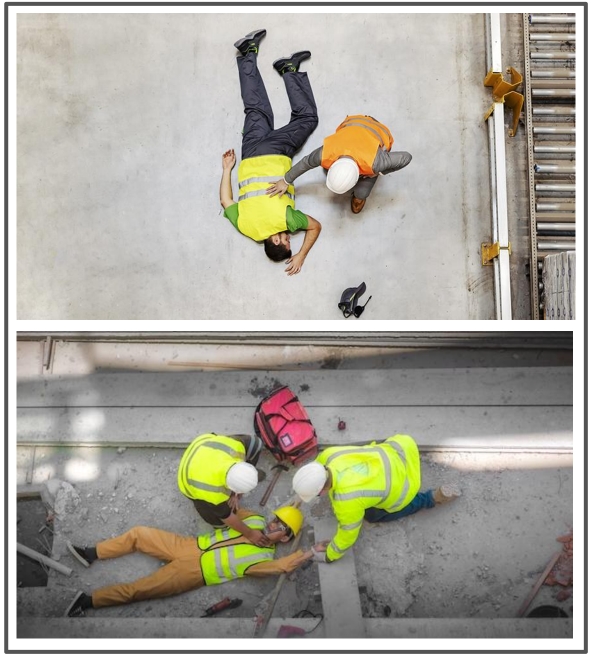
Fall Detection: A Review of Recent Research Papers

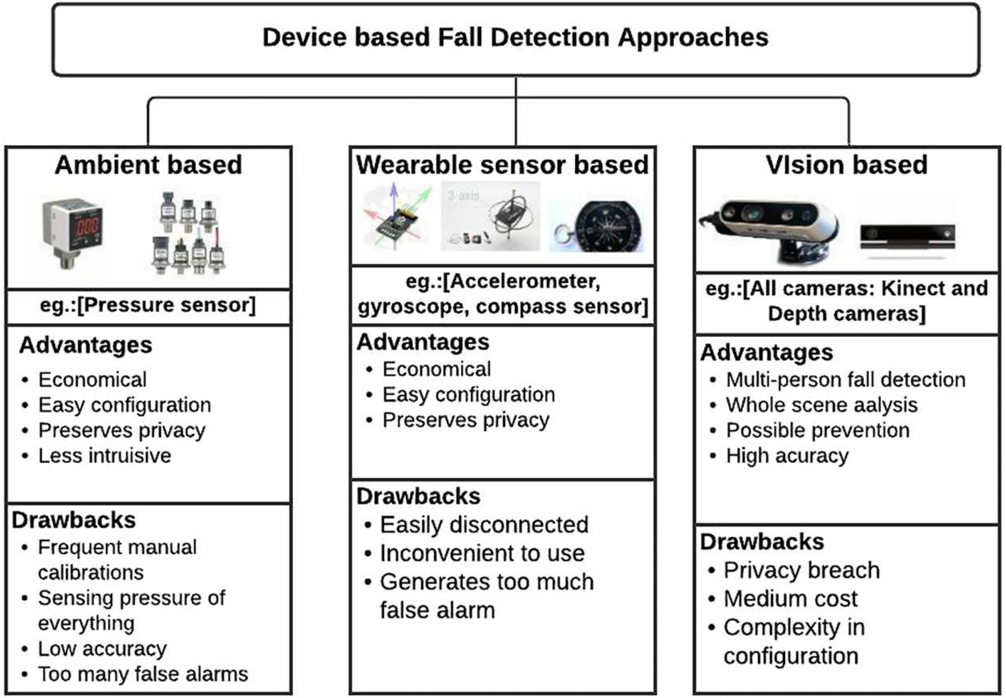
Abbos Aliboev

충북대학교
CHUNGBUK NATIONAL UNIVERSITY

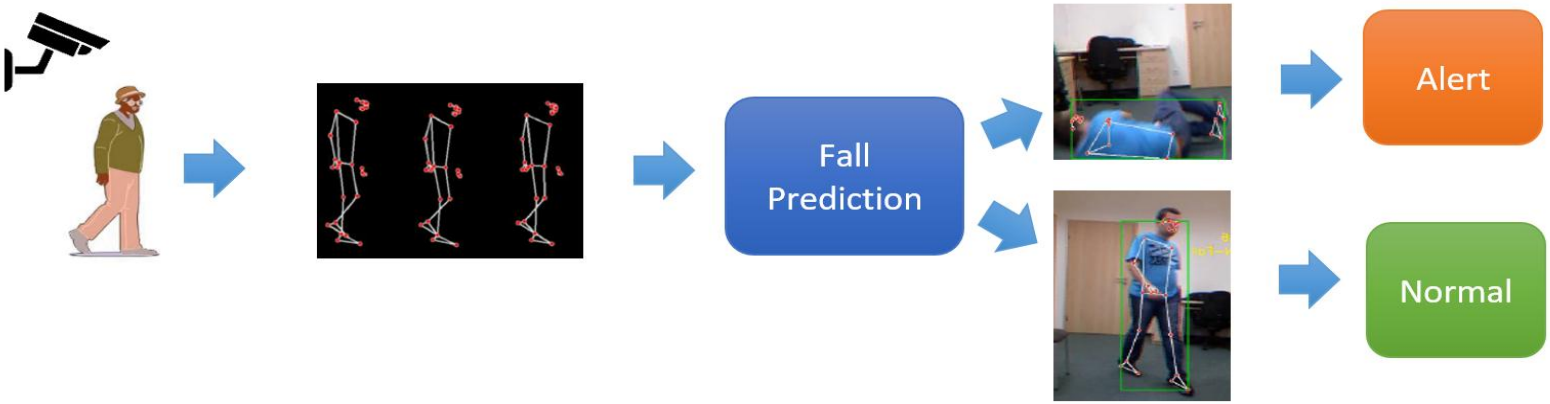
TABLE OF CONTENTS

Introduction





Simple Vision Based Example



01

Paper

Fall-portent detection for construction sites based on computer vision and machine learning.

Xiaoyu Liu, Feng Xu and Zhipeng Zhang
School of Naval Architecture, Ocean and Civil Engineering, Shanghai Jiao Tong University, Shanghai, China, and
Kaiyu Sun
Shanghai Jianke Engineering Consulting Co., Ltd, Shanghai, China

Received 14 May 2023
Revised 30 July 2023
30 August 2023
Accepted 13 September 2023

emerald PUBLISHING

Problems:

- Falls are common and dangerous at construction sites.
- Prior works: binary classification (fall / no-fall), often sensor-based.

Main Idea:

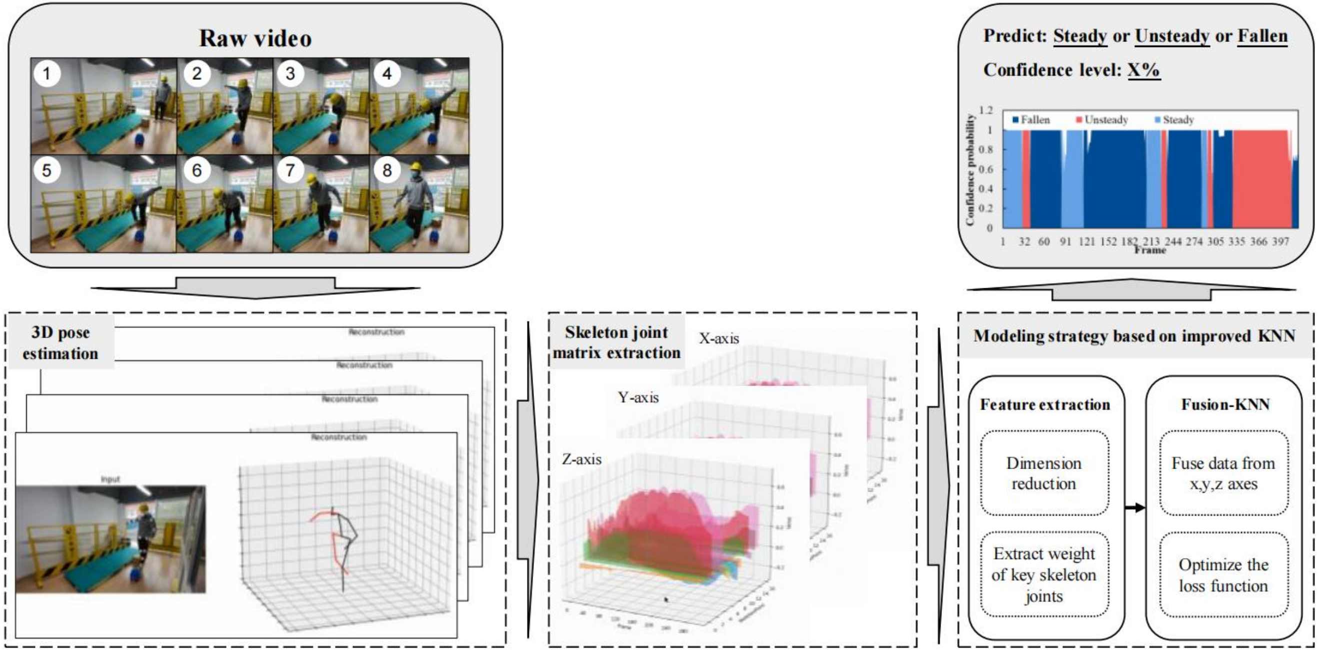
3-stage fall detection model

- **Steady** – standing normally
- **Unsteady** – balance lost
- **Fallen** – fall occurred

https://www.emerald.com/insight/content/doi/10.1108/ecam-05-2023-0458/full/html

Method Details

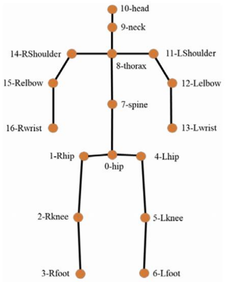
- Architecture of the methodology



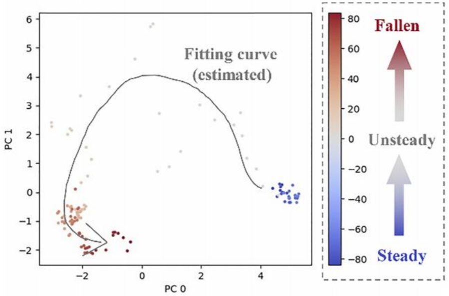
Video → 3D Pose Estimation (VideoPose3D) → Feature Selection (PCA + K-means) → Fusion-KNN → 3-Class Prediction

Step-by-Step Pipeline Explanation

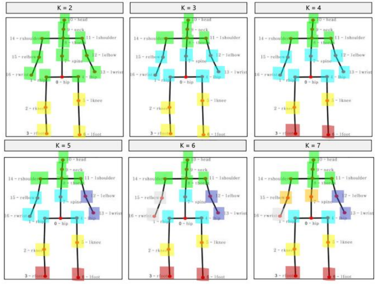
VideoPose3D	Converts each video frame into 17 keypoint 3D skeleton data (x, y, z)
PCA – Principal Component Analysis	PCA removes redundant and less relevant keypoints
K means – Clustering	K-means helps identify the most influential joints (e.g., hips, knees)
Fusion KNN – k Nearest Neighbors	Computes weighted distances (x/y/z) → emphasizes z-axis Classifies state as: Steady / Unsteady / Fallen
Post-processing:	Reclassifies short unsteady frames (< 6) as steady → Reduces false alarms, improves stability



Serial numbers of 17 joints



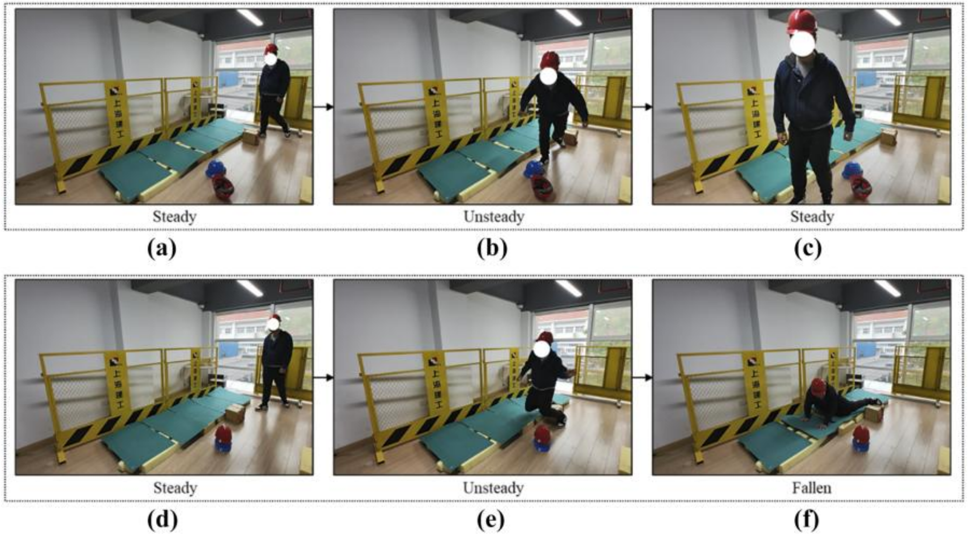
Visualization of banana trip by PCA



Experimantal Results of K-means clustering

Dataset Details

Samples of workers three states



Class balance:

- Steady: **688** frames
- Unsteady: **400** frames
- Fallen: **600** frames

Total frames: **1688**

Data type: 3D skeleton keypoints

Collected from: Simulated fall scenarios

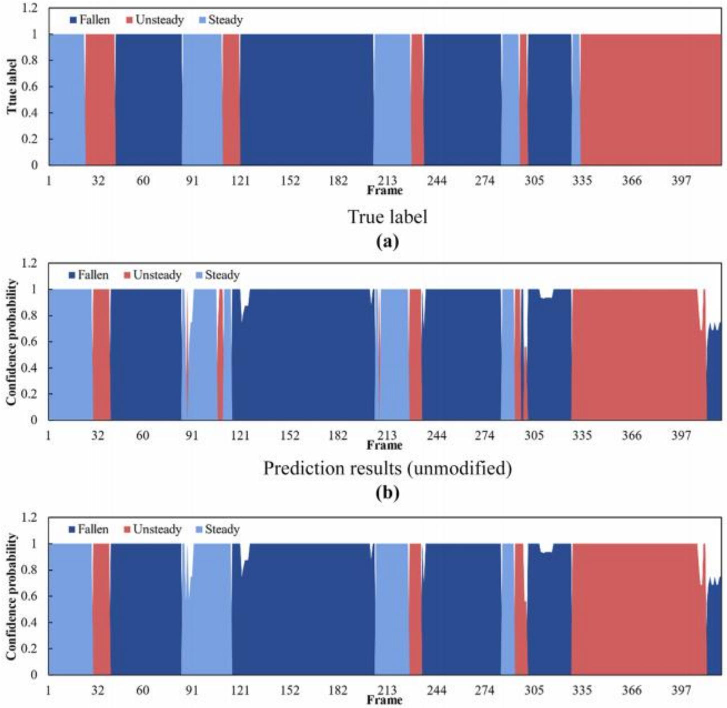
Results

		Predicted class		
True class		Steady	Unsteady	Fallen
	Steady	302	96	2
	Unsteady	74	456	70
	Fallen	31	45	612

Before filtering by threshold
(accuracy = 81.40%)
(a)

		Predicted class		
True class		Steady	Unsteady	Fallen
	Steady	336	64	0
	Unsteady	60	478	58
	Fallen	31	39	618

After filtering by threshold
(accuracy = 85.02%)
(b)



True label
(a)

Prediction results (unmodified)
(b)

Prediction results (after filtering by the threshold-based method)
(c)

Strengths & Limitations

Strengths	Limitations
3-class detection (Steady, Unsteady, Fallen)	Only 3.125 FPS (not real-time)
No sensors required – only camera input	Small, simulated dataset (1688 frames)
Simple, interpretable Fusion-KNN model	No real-world or multi-person testing
Emphasis on z-axis improves fall recognition	Not tested on real-world construction sites
Post-processing reduces false alarms	No deep learning or end-to-end training

02

Paper

A Real-time skeleton-based fall detection algorithm based on temporal convolutional networks and transformer encoder

ELSEVIER

Xiaoqun Yu, Chenfeng Wang, Wenyu Wu, Shuping Xiong

a. Department of Mechanical and Industrial Design, School of Mechanical Engineering, Southeast University, Nanjing 211189, PR China
b. Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34141, Republic of Korea

Received 9 August 2024, Revised 12 January 2025, Accepted 20 January 2025, Available online 20 January 2025, Version of Record 22 January 2025

This work was supported by the Basic Science Research Program of the National Research Foundation of Korea , Samsung Display Co., Ltd, and SEU start-up fund.

Problems:

- Occlusions (partially hidden bodies)
- Multiple people in the same frame
- Low performance on edge devices
- Imbalanced data (fewer fall samples)

Main Idea:

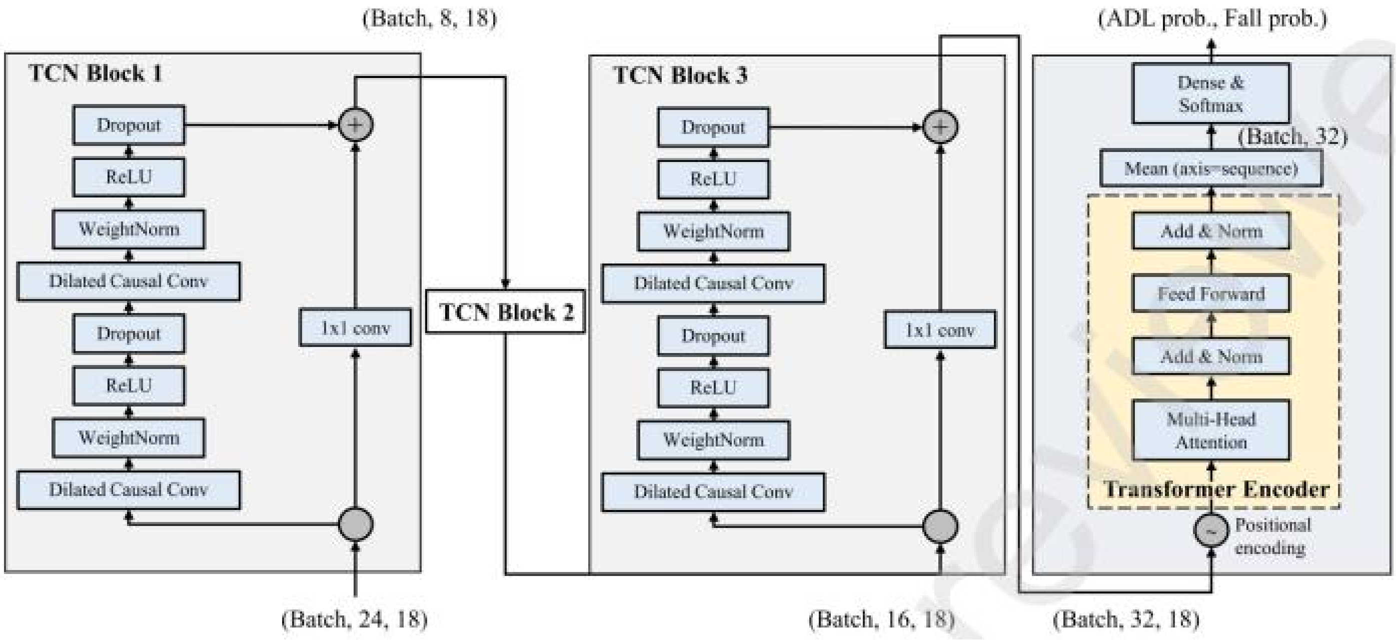
Skeleton-based Fall / Non-Fall detection using:

- Pose sequences
- TCN + Transformer
- Real-time edge deployment

https://doi.org/10.1016/j.pmcj.2025.102016

Method Details

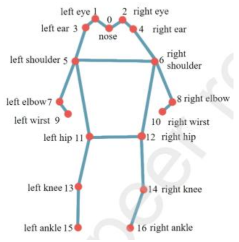
- The network architecture of TCNTE



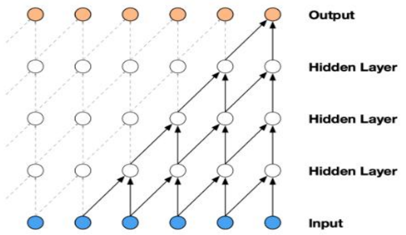
Video → YOLOv8-pose → BoT-SORT → Pose Normalization & Windowing → TCN + Transformer Encoder (TCNTE) → Classification

Step-by-Step Pipeline Explanation

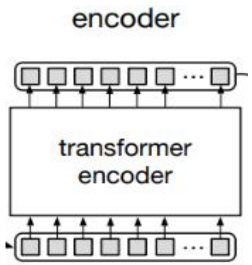
YOLOv8-pose	Detects humans and extracts 17 keypoint skeletons from each video frame
BoT-SORT	Tracks each person across frames, assigns consistent ID for each sequence
Pose Normalization	Aligns all skeletons to a standard scale and reference point (e.g., hip-centered, min-max normalization)
Windowing	Groups consecutive frames (e.g., 45) into a motion sequence for temporal analysis
TCN + Transformer Encoder (TCNTE)	Learns both short- and long-term motion patterns to detect falls accurately



17 key points (YOLOv8)

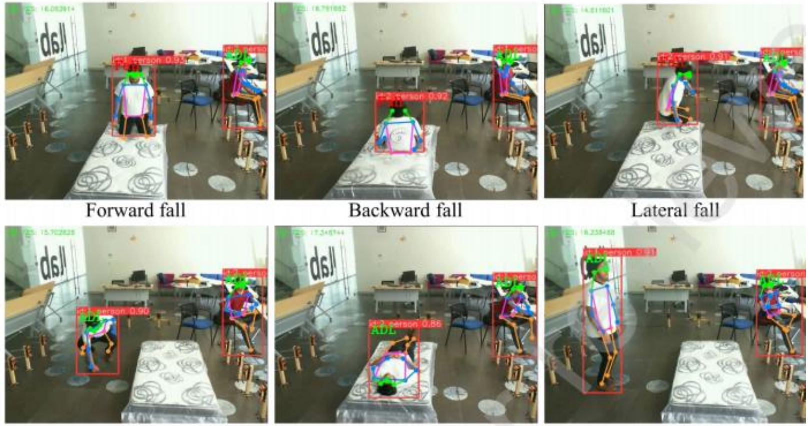


Temporal Convolution Network (TCN)



Transformer Encoder

Dataset Details



Forward fall | Backward fall | Lateral fall

Picking | Lying | Jumping

Classification results on UP-Fall test set using Jetson Orin NX

Table 1. Common public video-based fall datasets

Dataset	Types of ADLs[a]/falls	No. subjects	No. ADL/fall video clips	Video resolution
Le2i [33]	6/3	9	48/143	640x480, 25 fps
UR Fall [34]	NA	5	40/30	640x640, 30 fps
MultiCam [35]	NA	1	[b]24x8	720x480, 30 fps
UP-Fall [16]	6/5	17	306/255	640x480, 18 fps

a: ADLs (activities of daily living)
b: MultiCam dataset used 8 cameras and ADL and fall motions are mixed in one video clip

Table 5. Comparison with state-of-the-art fall detection algorithms using UP-Fall dataset

Study (year)	Algorithm	Sensitivity (%)	Specificity (%)	Accuracy (%)	*No. of parameters	Practical application
Espinosa, et al. [36] (2019)	CNN	97.75	83.08	95.64	268,674	N/A
Ramirez, et al. [13] (2021)	Random forest	98.82±0.03	99.48±0.05	99.34±0.03	N/A	N/A
Inturi, et al. [15] (2022)	1D-CNN+LSTM	94.37	98.96	98.59	N/A	N/A
Suarez, et al. [44] (2022)	1D-CNN	83.41	N/A	89.30	N/A	N/A
Zahan, et al. [19] (2023)	SGCN+Sep-TCN	93.33	84.40	87.61	340,000	N/A
Raza, et al. [45] (2023)	Vision transformer	97.15	N/A	97.36	N/A	N/A
Qi, et al. [42] (2023)	2D-CNN (vision)	97.71±0.28	N/A	97.56±0.20	N/A	Federated learning (server)
	2D-CNN (vision+IMU)	100±0.00		99.93±0.04		
Our study	**TCNTE**	**95.39±1.18**	**99.66±0.20**	**99.58±0.18**	**14,746**	**Jetson Orin NX (edge)**

Results

Table 3. classification results of three deep learning models on the test set.

Model	Window-wise			File-wise		
	Sensitivity	Specificity	Accuracy	Sensitivity	Specificity	Accuracy
TCN	96.03±1.05	99.18±0.23	99.12±0.22	99.45±0.77	88.94±3.90	93.01±2.85
TE	**96.69±0.61**	98.92±1.03	98.88±1.01	99.45±0.77	94.73±2.39	96.56±1.75
TCNTE	95.39±1.18	**99.66±0.20**	**99.58±0.18**	**99.45±0.77**	**96.77±2.28**	**97.81±1.58**

Table 4. Real-time performance evaluation in Jetson Orin NX

Model	Inference time (ms)	No. of parameters	fps (1 person)	fps (2 people)
TCN	5.7	7,706	20	18
TE	3.7	4,570	21	19
TCNTE	8.0	14,746	19	17

Table 6. Comparison of model performance using different loss functions

Loss function	Sensitivity (%)	Specificity (%)	Accuracy (%)
Cross entropy	0±0.00	100±0.00	98.17±0.00
Weighted cross entropy	96.60±0.70	97.92±1.02	97.90±0.99
Focal loss	92.74±1.94	99.60±0.29	99.48±0.28
Weighted focal loss	95.39±1.18	99.66±0.20	99.58±0.18

Strengths & Limitations

Strengths	Limitations
High accuracy (99.58%) and robust to noise	Dependent on pose estimation quality (YOLOv8-pose)
Real-time performance (19 FPS on Jetson Orin NX)	Needs consistent lighting and camera placement
Works in multi-person and occlusion-heavy scenes	Transformer adds computational cost compared to LSTM/CNN
No wearable sensors required — camera-only system	Tested only on indoor dataset — may need outdoor generalization
Uses strong architecture: TCN + Transformer	

03

Paper

Hybrid Deep Learning for Human Fall Detection: A Synergistic Approach Using YOLOv8 and Time-Space Transformers

YOUSEF SANJALAWE[1], SALAM FRAIHAT[2], MOSLEH ABUALHAJ[3], SALAM R. AL-E'MARI[4], AND EMRAN ALZUBI[5]

[1]Department of Information Technology, King Abdullah II School for Information Technology, The University of Jordan (JU), Amman 11942, Jordan
[2]Artificial Intelligence Research Center (AIRC), College of Engineering and Information Technology, Ajman University, Ajman, United Arab Emirates
[3]Department of Networks and Information Security, Faculty of Information Technology, Al-Ahliyya Amman University, Amman 19328, Jordan
[4]Information Security Department, Faculty of Information Technology, University of Petra, Amman 11196, Jordan
[5]College of Business Administration, Northern Border University (NBU), Arar 91431, Saudi Arabia

IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 7 February 2025, accepted 2 March 2025, date of publication 4 March 2025, date of current version 12 March 2025.

Problems:

- Old models are slow and can't run in real-time
- Many use sensors or need manual feature design
- Transformers are strong, but hard to train and slow

Main Idea:

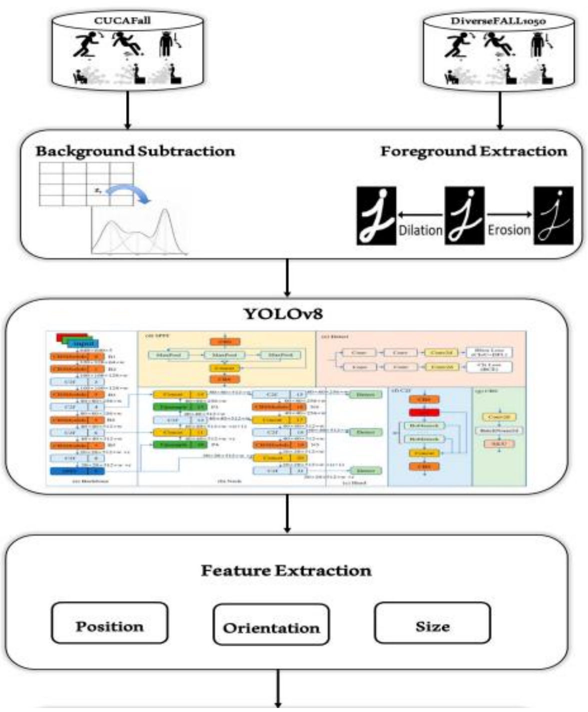
Combine YOLOv8 and a Time-Space Transformer (TST)

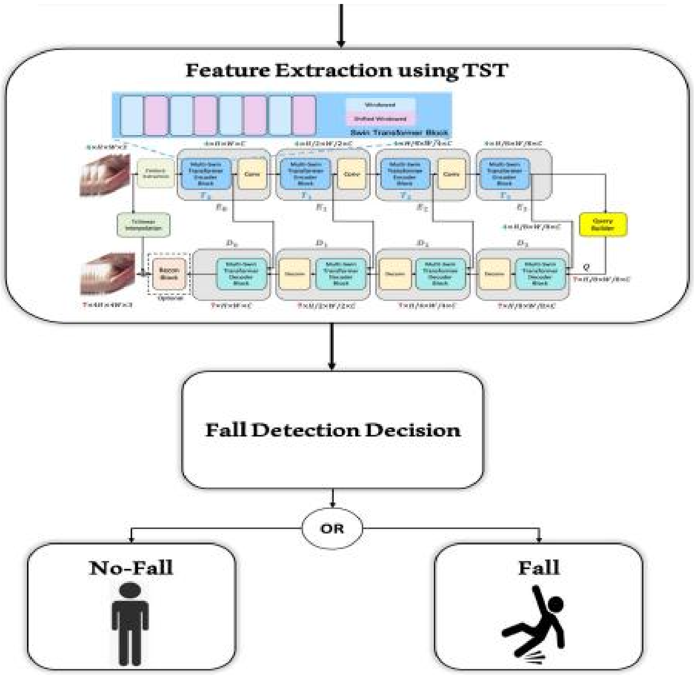
- YOLOv8 finds people in each video frame
- TST understands how they move over time
- Detects Fall vs Non-Fall

https://ieeexplore.ieee.org/abstract/document/10909442

Method Details

- Proposed methodology

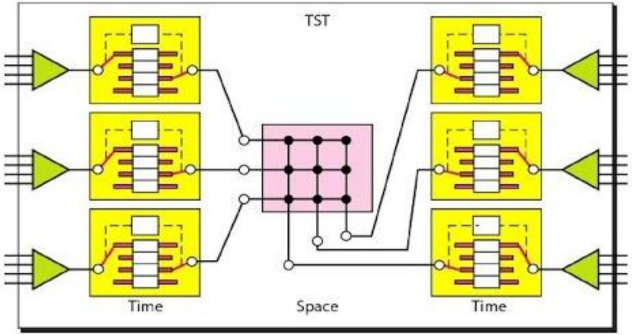




Video → **YOLOv8** → **CNN Features** → **Time-Space Transformer (TST)** → **Fall / Non-Fall Classification**

Step-by-Step Pipeline Explanation

YOLOv8	Detects humans in each frame and extracts bounding box features
CNN Feature Extraction	Converts each frame into a compact feature vector (appearance info)
Pose Normalization	Aligns all skeletons to a standard scale and reference point (e.g., hip-centered, min-max normalization)
Time-Space Transformer (TST)	Learns motion and time-based changes across frames. It looks at how the body moves in space, and how it changes over time.



Time Space Transformer (TST) - Time + Spce

Dataset Details

FIGURE 3. Sample of DiverseFALL10500 dataset.

FIGURE 4. Sample of CAUCAFall dataset.

TABLE 7. Dataset statistics before and after class balancing.

Dataset	Before Balancing		After Enhanced Balancing	
	Fall Instances	Non-Fall Instances	Fall Instances	Non-Fall Instances
CUCAFall	2,800 (35%)	5,200 (65%)	4,800 (48%)	5,200 (52%)
DiverseFALL10500	3,150 (30%)	7,350 (70%)	5,000 (45%)	6,000 (55%)

Results

TABLE 9. Quantitative analysis using benchmark DiverseFALL10500 dataset.

S No	Model	mAP	Accuracy	Precision	F1-score	Recall (TPR)	FPR	AUC-PR
1	Faster R-CNN	0.831	0.823	0.837	0.813	0.809	0.163	0.82276
2	yolov3	0.842	0.828	0.848	0.827	0.809	0.152	0.82804
3	yolov4	0.825	0.806	0.818	0.801	0.794	0.182	0.80582
4	yolov5n	0.870	0.846	0.840	0.839	0.852	0.160	0.84596
5	yolov5s	0.906	0.869	0.895	0.878	0.845	0.105	0.86928
6	yolov5m	0.848	0.825	0.850	0.825	0.807	0.150	0.82815
7	yolov5l	0.858	0.825	0.805	0.813	0.837	0.195	0.82067
8	yolov5x	0.834	0.800	0.769	0.797	0.829	0.231	0.79804
9	yolov8n	0.863	0.823	0.810	0.822	0.837	0.190	0.82347
10	YOLOv8S	0.910	0.865	0.890	0.879	0.851	0.110	0.86945
11	yolov8m	0.929	0.860	0.880	0.868	0.851	0.120	0.86363
12	yolov8l	0.871	0.855	0.855	0.856	0.845	0.145	0.85036
13	yolov8x	0.903	0.862	0.885	0.874	0.853	0.115	0.86393
14	Visionary vigilance [57]	0.935	0.884	0.900	0.884	0.859	0.100	0.88064
15	**Proposed**	**0.950**	**0.890**	**0.920**	**0.910**	**0.900**	**0.080**	**0.90976**

TABLE 10. Statistical analysis of DiverseFALL10500 model performance.

Metric	Value
Mean Accuracy	0.844
Confidence Interval (95%)	(0.829, 0.859)
T-Statistic	-6.427
P-Value	0.000016

TABLE 11. Quantitative analysis using benchmark CUCAFall dataset.

S No	Model	mAP	Accuracy	Precision	F1-score	Recall (TPR)	FPR	AUC-PR
1	Faster R-CNN	0.9903	0.9907	0.9891	0.9902	0.9924	0.01090	0.99075
2	yolov3	0.9912	0.9917	0.9904	0.9916	0.9931	0.00960	0.99175
3	yolov4	0.9896	0.9907	0.9901	0.9898	0.9913	0.00990	0.99070
4	yolov5n	0.9924	0.9934	0.9921	0.9935	0.9947	0.00790	0.99340
5	yolov5s	0.9932	0.9952	0.9943	0.9954	0.9961	0.00570	0.99520
6	yolov5m	0.9911	0.9925	0.9914	0.9923	0.9927	0.00860	0.99205
7	yolov5l	0.9926	0.9938	0.9925	0.9934	0.9942	0.00750	0.99335
8	yolov5x	0.9898	0.9905	0.9892	0.9903	0.9906	0.01080	0.98990
9	yolov8n	0.9925	0.9930	0.9921	0.9923	0.9935	0.00790	0.99280
10	YOLOV8S	0.9950	0.9964	0.9977	0.9952	0.9978	0.00230	0.99775
11	yolov8m	0.9949	0.9964	0.9948	0.9952	0.9978	0.00520	0.99630
12	yolov8l	0.9947	0.9932	0.9941	0.9914	0.9950	0.00590	0.99455
13	yolov8x	0.9945	0.9939	0.9937	0.9941	0.9932	0.00630	0.99345
14	Visionary vigilance [57]	0.9954	0.9983	0.9982	0.9981	0.9984	0.00180	0.99830
15	**Proposed**	**0.9998**	**0.9988**	**0.999**	**0.9988**	**0.9986**	**0.00100**	**0.99880**

TABLE 12. Statistical analysis of CUCAFall model performance.

Metric	Value
Mean Accuracy	0.9939
Confidence Interval (95%)	(0.9924, 0.9954)
T-Statistic	-7.1743
P-Value	0.000005

Strengths & Limitations

Strengths	Limitations
Very high accuracy: Accuracy ~99.84%, F1-score ~99.82	Transformer needs more memory (computationally expensive)
Learns both space (YOLOv8) and time (Transformer)	Depends on good camera input
Works without sensors	Transformer adds computational cost compared to LSTM/CNN
Can detect multiple people	Tested in limited environments
Uses strong architecture: TCN + Transformer	

Conclusion

& Insight

Conclusion & Insights

Paper	Main Method	Key Feature
Paper 1	PCA + K-means + Fusion-KNN	3D skeleton, 3-stage state classification
Paper 2	YOLOv8 + BoT-SORT + TCN + Transformer	Real-time fall detection, occlusion handling
Paper 3	YOLOv8 + Time-Space Transformer	High accuracy using pure visual input

What works well:

• High accuracy in controlled environments

• Real-time detection using lightweight models

• Useful combination of spatial (YOLO) and temporal (Transformer) features

What is missing:

• Most papers are tested only indoors

• Construction site scenarios with many workers, gear, and occlusion are not fully addressed

THANKS!

Q&A

Abbos Aliboev

GitHub: https://github.com/abbosaliboev/fall-detection-research/